

stm32f103 workspace notes

FreeRTOS Semaphore Types

Four types, one underlying queue mechanism. They differ in the rules the kernel enforces — ownership, count ceiling, and priority inheritance.

Binary 0 or 1 Signalling between tasks or ISR → task No owner No inheritance ISR-safe	Mutex 0 or 1, owned Protecting a shared resource Owner enforced Priority inheritance Not ISR-safe	Recursive Mutex 0 to N, owned Re-entrant code that locks the same resource Owner enforced Priority inheritance Not ISR-safe	Counting 0 to maxCount Resource pools or accumulating events No owner No inheritance ISR-safe
---	---	---	---

All four are created via the `semphr.h` API and held as a `SemaphoreHandle_t`. The underlying kernel object is the same queue structure — only the creation call and enforcement rules differ.

1 Binary Semaphore

```
XSEMAPHORECREATEBINARY()
```

A single token — either available or not. No ownership: any task (or ISR) can give it, and any task can take it. The typical use is **signalling**: one side announces that something happened, the other side waits for that announcement before proceeding.

Simple model: think of a doorbell. The person outside presses it (give). The person inside hears it and opens the door (take). The bell doesn't care who pressed it or who answered — it just records that it was pressed once.

Example — ISR signals a task that a UART byte arrived

```
static SemaphoreHandle_t xByteReady;

/* ISR - runs in Handler mode when UART receives a byte */
void USART1_IRQHandler(void)
{
    BaseType_t xWoken = pdFALSE;
    xByteReceived = USART1->DR;          /* read the byte */
    xSemaphoreGiveFromISR(xByteReady, &xWoken); /* signal the task */
    portYIELD_FROM_ISR(xWoken);
}

/* Task - processes the byte whenever the ISR says one arrived */
```

```

static void uart_rx_task(void *pv)
{
    for (;;) {
        xSemaphoreTake(xByteReady, portMAX_DELAY); /* block until ISR gives */
        process(xByteReceived);                    /* safe to read now */
    }
}

int main(void)
{
    xByteReady = xSemaphoreCreateBinary();          /* starts NOT available */
    xTaskCreate(uart_rx_task, "rx", 128, NULL, 2, NULL);
    vTaskStartScheduler();
}

```

Notice `xSemaphoreGiveFromISR()` rather than `xSemaphoreGive()` — ISR-safe variants exist for both binary and counting semaphores. Mutexes have no ISR variant at all because priority inheritance cannot work inside an ISR.

2 Mutex

`XSEMAPHORECREATEMUTEX()`

Structurally a binary semaphore, but with two extra rules: **ownership** (only the task that took it may give it back) and **priority inheritance** (if a higher-priority task blocks on it, the holder is temporarily boosted to the waiter's priority so it can finish quickly). Starts available — no pre-give needed. Use it to protect shared resources.

Simple model: a toilet key on a large ring at a petrol station. You take the key, go in, come back, and hand it back yourself — someone else can't return it for you. If the manager (high priority) is queuing behind a customer (low priority) who has the key, someone had better hurry up.

Example — two tasks share a UART transmit function

```

static SemaphoreHandle_t xUartMutex;

void uart_print(const char *s)
{
    xSemaphoreTake(xUartMutex, portMAX_DELAY); /* acquire */
    while (*s) {
        while (!(USART1->SR & USART_SR_TXE)) {}
        USART1->DR = *s++;
    }
    xSemaphoreGive(xUartMutex);                /* release - SAME task */
}

static void sensor_task(void *pv)
{

```

```

    for (;;) {
        uart_print("sensor reading\r\n");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

static void status_task(void *pv)
{
    for (;;) {
        uart_print("status ok\r\n");
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

int main(void)
{
    xUartMutex = xSemaphoreCreateMutex();      /* starts available */
    xTaskCreate(sensor_task, "sens", 128, NULL, 1, NULL);
    xTaskCreate(status_task, "stat", 128, NULL, 2, NULL);
    vTaskStartScheduler();
}

```

Without the mutex, a context switch mid-print would interleave the two tasks' characters into garbled output. The mutex ensures one task's call to `uart_print()` runs to completion before the other starts.

3 Recursive Mutex

`XSEMAPHORECREATERECURSIVEMUTEX()` · `XSEMAPHORETAKERECURSIVE()` · `XSEMAPHOREGIVERECURSIVE()`

A mutex the **same task** can take multiple times without deadlocking itself. Each take increments an internal count; the resource is only truly released when an equal number of gives have returned the count to zero. Same ownership and priority inheritance as a regular mutex.

When you need this: a function that holds a lock calls another function that tries to take the same lock. With a plain mutex that would deadlock immediately — the first take succeeded, the second one blocks forever waiting for the holder (itself). A recursive mutex lets the same owner nest takes freely.

Example — nested calls both lock the same resource

```

static SemaphoreHandle_t xBusMutex;

/* low-level: send one I2C byte — takes the bus mutex */
void i2c_write_byte(uint8_t b)
{
    xSemaphoreTakeRecursive(xBusMutex, portMAX_DELAY); /* take #1 (or #2) */
    i2c_hw_send(b);
}

```

```

    xSemaphoreGiveRecursive(xBusMutex);          /* give #1 (or #2) */
}

/* high-level: write a register - ALSO takes the bus mutex */
void sensor_write_reg(uint8_t reg, uint8_t val)
{
    xSemaphoreTakeRecursive(xBusMutex, portMAX_DELAY); /* take #1 */
    i2c_write_byte(reg);    /* takes AGAIN (#2) -- would deadlock with plain mutex! */
    i2c_write_byte(val);    /* takes AGAIN (#2) -- same */
    xSemaphoreGiveRecursive(xBusMutex);          /* give #1 - count back to 0 */
}                                              /* bus truly released here */

int main(void)
{
    xBusMutex = xSemaphoreCreateRecursiveMutex();
    /* ... create tasks ... */
    vTaskStartScheduler();
}

```

The count inside the recursive mutex: after `sensor_write_reg()` takes once (count=1), the first `i2c_write_byte()` takes again (count=2), gives (count=1), the second `i2c_write_byte()` takes (count=2), gives (count=1), then `sensor_write_reg()` gives (count=0, bus released). Other tasks can only enter after that final give.

4 Counting Semaphore

`XSEMAPHORECREATECOUNTING(UXMAXCOUNT, UXINITIALCOUNT)`

Like a binary semaphore but the token count can rise above 1, up to a configured maximum. No ownership. Two distinct use-cases with opposite initial counts:

- **Resource pool** — initialise to N (the pool size). Each take claims one resource, each give returns one. Blocks when the pool is exhausted.
- **Event accumulation** — initialise to 0. An ISR or producer gives once per event; a consumer takes once per event. If the producer runs faster than the consumer, the count accumulates rather than losing events the way a binary semaphore would.

Simple model (resource pool): a car park with 3 spaces and a display showing "spaces available." Each car entering decrements the count; each car leaving increments it. A fourth car blocks at the barrier until someone leaves.

Example A — resource pool (3 SPI chip-selects, multiple tasks share them)

```

/* 3 CS lines available; start at maximum */
static SemaphoreHandle_t xCSPool;

void spi_transaction(void)

```

```

{
    xSemaphoreTake(xCSPool, portMAX_DELAY); /* claim one CS slot */
    spi_do_work();
    xSemaphoreGive(xCSPool);                /* return the slot */
}

int main(void)
{
    xCSPool = xSemaphoreCreateCounting(3, 3); /* max=3, initial=3 */
    /* ... */
}

```

Example B – event accumulation (ISR fires faster than the task can consume)

```

static SemaphoreHandle_t xAdcEvents;

/* ADC conversion-complete ISR - fires frequently */
void ADC1_2_IRQHandler(void)
{
    BaseType_t xWoken = pdFALSE;
    adc_store_sample(ADC1->DR);
    xSemaphoreGiveFromISR(xAdcEvents, &xWoken); /* +1 to count */
    portYIELD_FROM_ISR(xWoken);
}

/* processing task - takes one per event, never misses any */
static void adc_process_task(void *pv)
{
    for (;;) {
        xSemaphoreTake(xAdcEvents, portMAX_DELAY); /* -1 from count */
        process_one_sample();
    }
}

int main(void)
{
    xAdcEvents = xSemaphoreCreateCounting(20, 0); /* max=20, initial=0 */
    /* ... */
}

```

If the ISR fires 5 times while the task is busy with one sample, the count reaches 5. The task then drains it one by one without losing any events. A binary semaphore in the same situation would only record "at least one," losing the other four.

All four, side by side

TYPE	CREATE	COUNT	OWNER ENFORCED	PRIORITY INHERITANCE	ISR-SAFE GIVE	USE FOR
Binary	xSemaphoreCreateBinary()	0 or 1	No	No	Yes	Signalling, ISR→task handoff
Mutex	xSemaphoreCreateMutex()	0 or 1	Yes	Yes	No	Protecting shared resources
Recursive mutex	xSemaphoreCreateRecursiveMutex()	0 to N	Yes	Yes	No	Re-entrant code, nested locking
Counting	xSemaphoreCreateCounting(max, init)	0 to max	No	No	Yes	Resource pools, event accumulation